

Technical Take-Home Assignment

Coding task and system design task

Please submit your work before the deadline below.

Overview

Complete the coding task and the system design task, then send your submission by email. The follow-up video call will focus on discussing your decisions, tradeoffs, tests, and improvement ideas.

Take-home

AI tools allowed

Prompt log required

Discussion-based call

Company	Haiper-X
Deadline	31 May 2026
Submit to	info@haiper-x.com

Assignment Scenario

Haiper-X is exploring a real estate platform for monthly rentals and property sales. Users can browse listings, filter by budget and location, save interesting listings, and contact agents or owners.

Your submission has two parts:

1. A coding task: complete a small property listing filter and summary function.
2. A system design task: design a larger rental and sales marketplace.

The video interview is not a live coding test by default. We will review your submission before the call. During the call, we will ask you to explain your approach, clarify tradeoffs, and discuss how you would improve the solution.

Part 1: Coding Task

Complete the missing implementation in the Python starter code below. The main program, sample data, expected output shape, and tests are already defined. Please keep the public function names and return shape unchanged.

Area	Requirement
Input record	Each listing has id, title, district, listing_type, price, bedrooms, area_sqm, and available.
Listing types	rent and sale.
Filters	Match by listing type, optional district, maximum price, minimum bedrooms, and availability.
Output	Matching listing IDs, match count, average price, invalid record details, and counts by district.

Starter Code

Create a file named `property_filter.py`. You may add helper functions, but do not change the names or signatures of `filter_listings` and `run_tests`.

```
from __future__ import annotations
from typing import Any, Optional

LISTINGS: list[dict[str, Any]] = [
    {
        "id": "R-100",
        "title": "Studio near District 1",
        "district": "District 1",
        "listing_type": "rent",
        "price": 750,
        "bedrooms": 1,
        "area_sqm": 32,
        "available": True,
    },
    {
```

```

        "id": "R-101",
        "title": "Two bedroom apartment in Thao Dien",
        "district": "Thao Dien",
        "listing_type": "rent",
        "price": 1400,
        "bedrooms": 2,
        "area_sqm": 78,
        "available": True,
    },
    {
        "id": "S-200",
        "title": "Condo for sale in Binh Thanh",
        "district": "Binh Thanh",
        "listing_type": "sale",
        "price": 180000,
        "bedrooms": 2,
        "area_sqm": 70,
        "available": True,
    },
    {
        "id": "R-102",
        "title": "Unavailable rental listing",
        "district": "District 7",
        "listing_type": "rent",
        "price": 900,
        "bedrooms": 2,
        "area_sqm": 65,
        "available": False,
    },
    {
        "id": "BROKEN-1",
        "title": "Missing valid price",
        "district": "District 1",
        "listing_type": "rent",
        "price": "contact us",
        "bedrooms": 1,
        "area_sqm": 40,
        "available": True,
    },
]

```

```

def filter_listings(
    listings: list[dict[str, Any]],
    listing_type: str,
    max_price: float,
    district: Optional[str] = None,
    min_bedrooms: int = 0,
    only_available: bool = True,
) -> dict[str, Any]:
    """
    Return a summary dictionary with this exact shape:

    {
        "matching_ids": list[str],
        "match_count": int,
        "average_price": float | None,
        "counts_by_district": dict[str, int],
        "invalid_records": list[dict[str, str]],
    }

```

Rules:

- Accept listing_type "rent" or "sale" only.
- Skip invalid records and include {"id": <id>, "reason": <reason>}.
- A valid record must have id, district, listing_type, numeric price, integer bedrooms, numeric area_sqm, and boolean available.
- Match records with the requested listing_type.
- If district is provided, match it case-insensitively.
- Match records with price <= max_price and bedrooms >= min_bedrooms.
- If only_available is True, exclude unavailable listings.
- Keep matching_ids in the same order as the input.

```

- average_price is None when there are no matches.
- Do not mutate the input listings.
"""

# TODO: Implement this function.
# You can add helper functions above if it makes the code clearer.
raise NotImplementedError

def run_tests() -> None:
    result = filter_listings(
        LISTINGS,
        listing_type="rent",
        max_price=1000,
        district="district 1",
        min_bedrooms=1,
    )
    assert result["matching_ids"] == ["R-100"]
    assert result["match_count"] == 1
    assert result["average_price"] == 750
    assert result["counts_by_district"] == {"District 1": 1}
    assert result["invalid_records"] == [
        {"id": "BROKEN-1", "reason": "invalid price"}
    ]

    sale_result = filter_listings(
        LISTINGS,
        listing_type="sale",
        max_price=200000,
        min_bedrooms=2,
    )
    assert sale_result["matching_ids"] == ["S-200"]
    assert sale_result["average_price"] == 180000

    empty_result = filter_listings(
        LISTINGS,
        listing_type="rent",
        max_price=500,
    )
    assert empty_result["matching_ids"] == []
    assert empty_result["average_price"] is None

    print("All tests passed")

if __name__ == "__main__":
    run_tests()

```

Expected Engineering Work

- Include sample input data or a simple way to run the code.
- Include tests or a clear manual verification checklist.
- Document assumptions, such as how many days count as long-open.
- Do not include secrets, private company code, or real user data.

Part 2: System Design Task

Design a real estate marketplace for monthly rentals and property sales. The platform should support renters, buyers, agents, property owners, and internal operations staff. You may imagine product details freely, but your design should be realistic enough for a first production launch and a path to growth.

Product Context

Assume Haiper-X wants to launch in one large city first, then expand to multiple cities. The platform should handle verified listings, search and filtering, listing detail pages, saved listings, inquiries, agent/owner dashboards, and internal moderation.

Area	Assumption
Initial scale	50,000 active listings, 500,000 monthly visitors, and 5,000 agents or owners.
Growth target	Multi-city expansion, millions of listings over time, and high search traffic during evenings and weekends.
Listing types	Monthly rentals and sale listings. Each listing has location, price, photos, property details, availability, contact method, and verification status.
Important risks	Duplicate or fake listings, stale availability, slow search, spam inquiries, privacy issues, and inconsistent agent response quality.

Please cover:

- Main users and critical workflows.
- Listing creation, verification, search, detail view, saved listings, and inquiry flow.
- Core data model and key APIs.
- Search and filtering strategy for location, price, bedrooms, property type, and availability.
- Photo/media handling and listing moderation.
- Permissions for user, agent/owner, moderator, and admin roles.
- Availability freshness, duplicate detection, fraud/spam prevention, and privacy.
- Reliability, observability, caching, and failure handling.
- What you would launch first, what you would defer, and why.

AI Tool Usage

You may use AI coding agents or assistants, including tools such as Codex, Claude Code, Cursor, GitHub Copilot, and ChatGPT. AI usage is not a penalty.

We care about ownership and verification. If you use AI tools, include a `PROMPTS.md` file describing what you asked, what output you used, what you changed, and how you verified the result.

Your `PROMPTS.md` should include:

- Tool names and purpose of use.
- Important prompts or a clear summary of the prompt inputs.
- Files or design sections influenced by AI output.
- Changes you made after reviewing the output.
- Tests, review steps, or manual checks you used to verify correctness.

Submission

Send your submission as a private repository link, a zip file, or attachments. If you use a repository link, please make sure access is enabled before sending the email.

Item	What to include
README .md	How to run the code, how to run tests, assumptions, known limitations, and important files to review.
Coding solution	Source code, sample data, tests or verification notes.
Design solution	Design document and diagrams if useful.
PROMPTS .md	Required if you used AI tools.

Email Format

Subject	[Technical Assignment] [Your Name]
Body	Your name, repository link or attachment list, run/test commands, whether AI tools were used, and anything reviewers should know first.

Evaluation Criteria

We evaluate both the submitted work and the follow-up discussion.

Requirements	Clear interpretation of the problem, assumptions, and edge cases.	10
Correctness	Core behavior works and invalid inputs are handled intentionally.	20
Code Quality	Readable structure, maintainability, simple abstractions, and practical organization.	15
Testing	Useful automated tests or a strong manual verification process.	10
System Design	Clear data model, APIs, permissions, workflows, reliability, and scalability thinking.	20
Tradeoffs	Ability to explain alternatives, risks, costs, and why you chose your approach.	10
Communication	README, design notes, and interview explanations are easy to follow.	10
AI Transparency	Clear prompt log and evidence that you reviewed and verified AI-assisted work.	5

Follow-Up Call

During the video call, be ready to discuss:

- Your most important assumptions and tradeoffs.
- Code structure, data flow, validation, and tests.
- Design choices, bottlenecks, failure scenarios, and monitoring.
- What you would improve with more time.
- How you used AI tools and how you verified the result.